ENTREGA 1: MÓDULO RS-232

Miembros del grupo: Héctor Carnicer Ull y César Alonso Romero Fecha: 05/10/2026

1) Introducción.

Se ha propuesto implementar un controlador con una interfaz a un sistema cliente para recibir y transmitir datos por una línea serie siguiendo el estándar RS232 simplificado.

Entre los aspectos configurables de este controlador se encuentran: velocidad de transmisión, número de bits de datos por palabra, existencia o no de bit de paridad y su tipo, número de bits de parada.
Para el caso concreto del trabajo: 8 bits de datos por palabra, sin bit de paridad, con 1 bit de parada. Transmitiéndose el bit menos significativo en primer lugar.
La posibilidad de modificar estos parámetros podría constituirse como mejora del controlador.

La solución que se ha propuesto se basa en la división en dos sistemas: de recepción y transmisión. Al funcionar de manera simultánea la comunicación será full-duplex.
El transmisor debe recibir un byte de datos del cliente a través de un bus paralelo asíncrono con señales de negociación y enviarlo inmediatamente, además de indicar que el sistema está listo u ocupado.
El receptor debe disponer de una interfaz síncrona y paralela de lectura de datos, además de capacidad de recibir múltiples de ellos y almacenarlos hasta que el cliente decida retirarlos. A parte, debe disponer de líneas que indiquen si existen datos por retirar y si el controlador ha agotado su memoria interna.

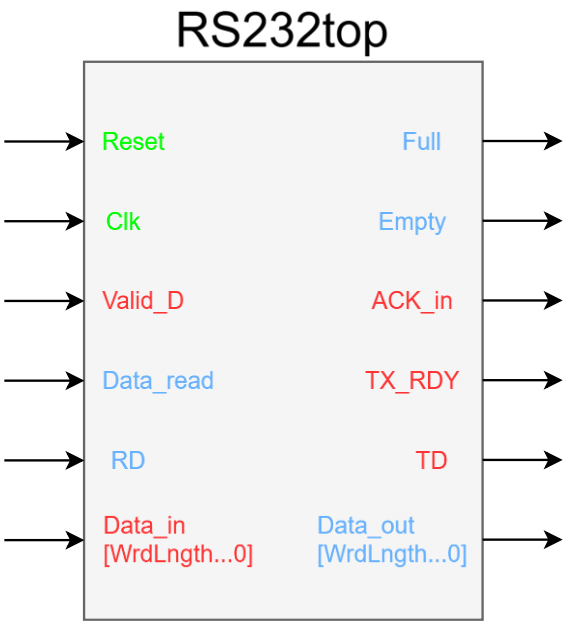
El sistema transmisor se implementa mediante una máquina de estados que construye la palabra detectando el bit de inicio, estableciendo el tamaño de bit determinado por la velocidad de transmisión y generando el bit de parada al completar el tamaño de palabra elegido.
El sistema receptor se implementa mediante una máquina de estados parecida a la del sistema transmisor, detectando el bit de inicio de los datos recibidos, leyendo los datos en el tiempo determinado por la velocidad de transmisión (a mitad de bit) e identificando el bit de parada, de tal manera que cada bit quede registrado en el registro de desplazamiento y al completarse la palabra, esta quede almacenada en la memoria interna, quedando a la espera de ser leída cuando se reciba la señal para ello.

2) Módulo RS-232

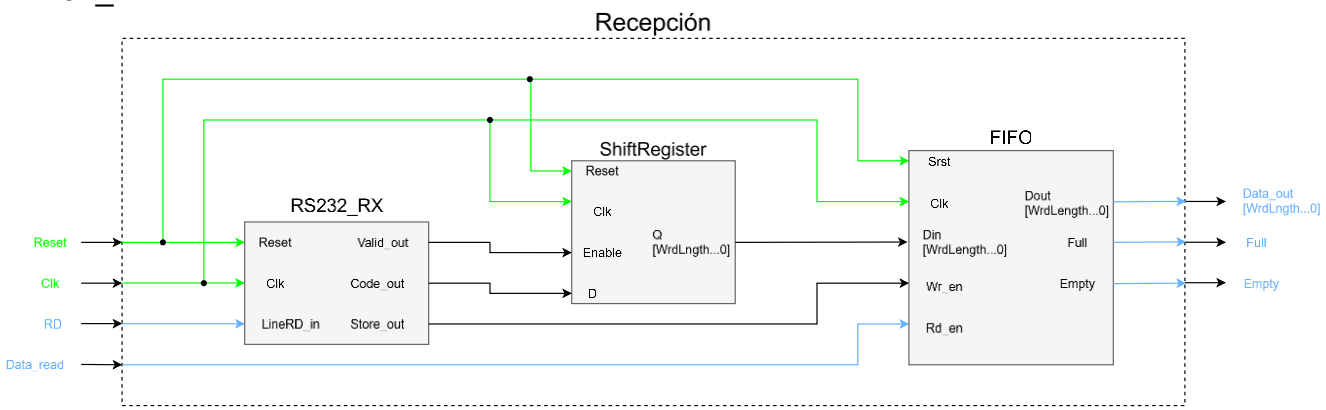
Retos técnicos superados del RS-232	Conseguido (Sí/No)
Implementación del RX en VHDL	Sí
Implementación del TX en VHDL	Sí
Simulación Behavioral	Sí
Simulación con retardos	No
Sistema RS232 funcionando en placa	No

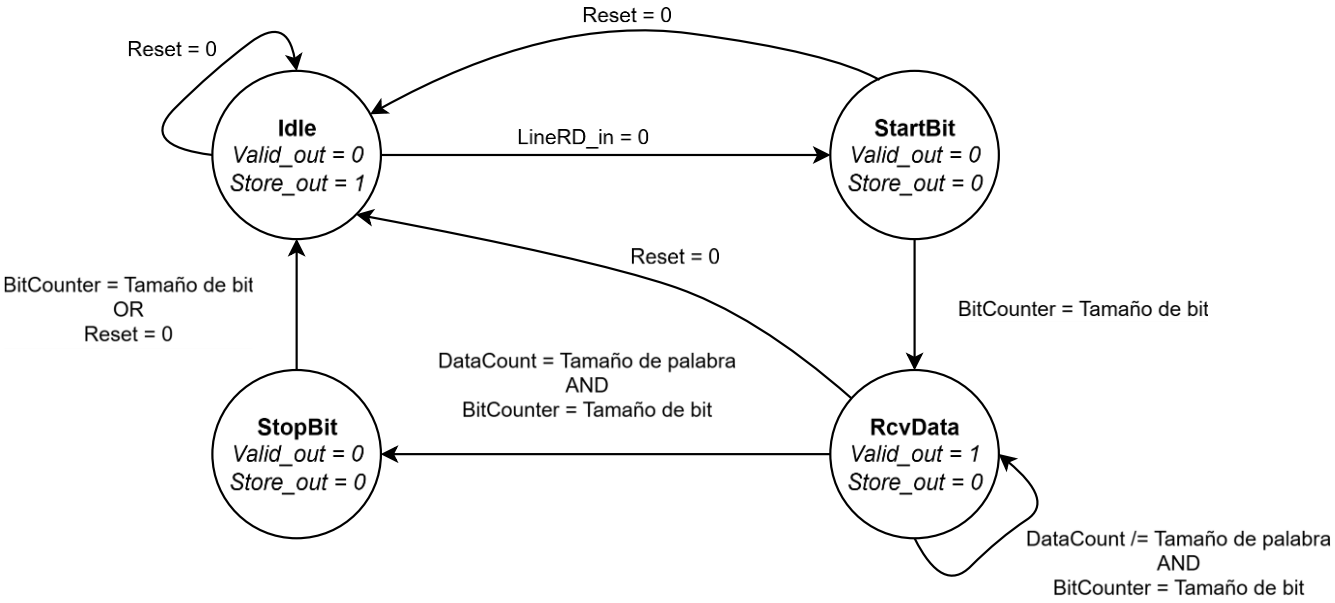
Mejoras del RS-232
Ninguna de momento, ver apartado 2.4

2.1) Descripción del sistema RS-232

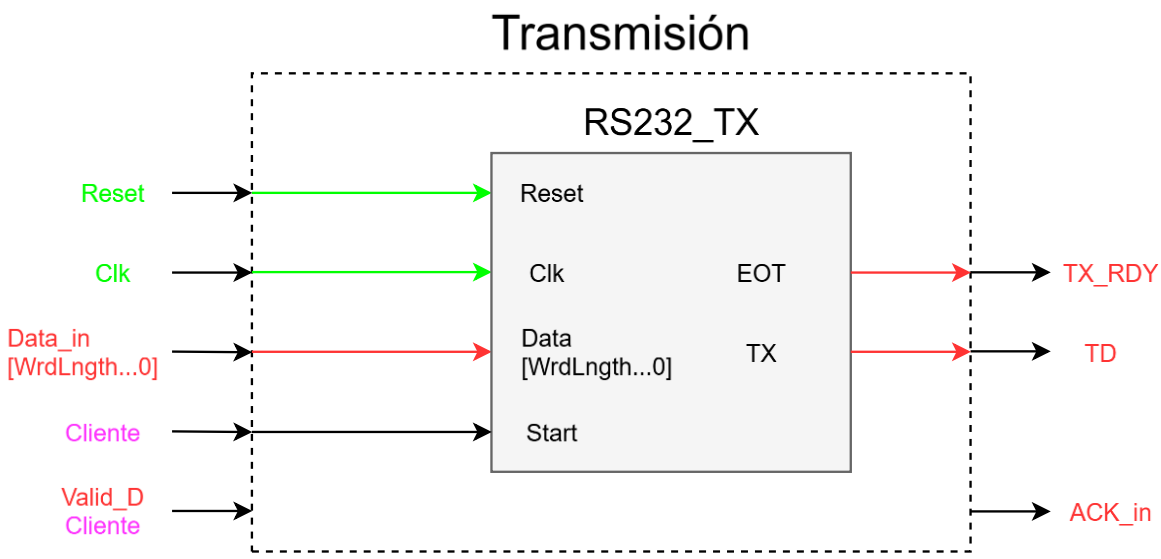


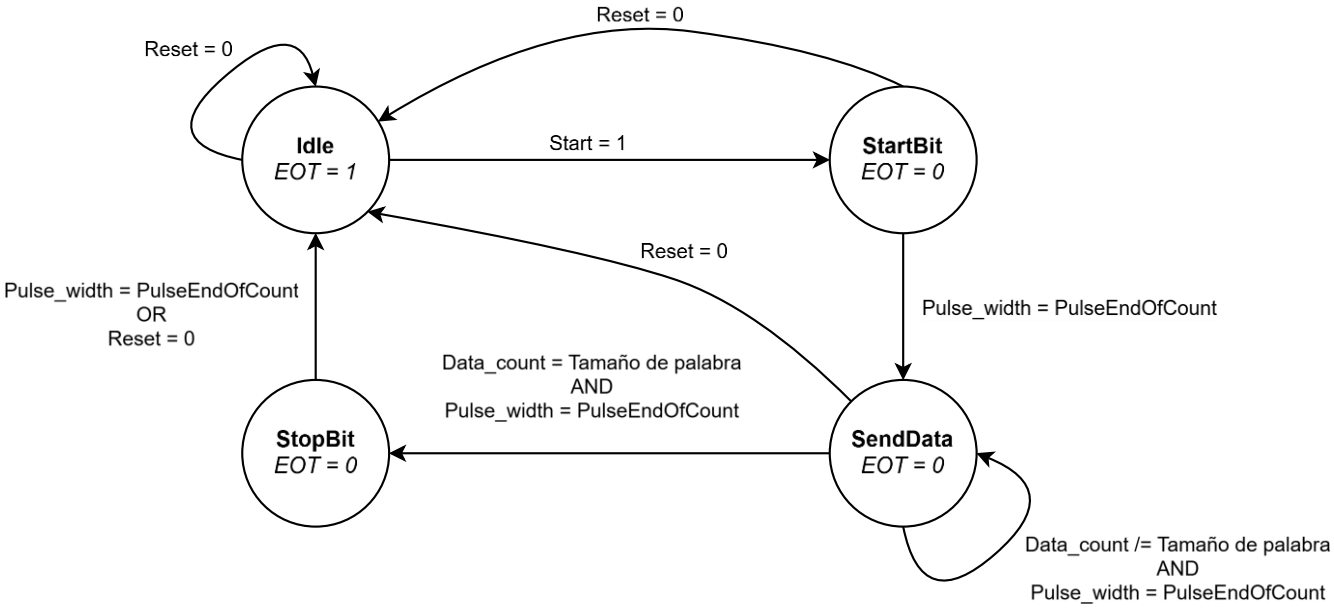
El sistema RS232top se compone de dos bloques: RS232_RX y RS232_TX.
En verde se muestran las señales comunes a ambos bloques, en azul las señales de entrada y salida del bloque RS232_RX y en rojo las señales de entrada y salida del bloque RS232_TX.





El bloque RS232_RX gestiona la recepción de datos del exterior hacia el cliente. Consta de reloj *Clk* y entrada *Reset* asíncrona. Se compone de tres sub-bloques: RS232_RX, ShiftRegister y FIFO.
El sub-bloque RS232_RX recibe los bits del exterior a través del puerto *LineRD_in*, si el bit muestreado es válido, activa la salida a nivel alto *Valid_Out* y pasa el bit a través de la salida *Code_Out*, que llega al sub-bloque ShiftRegister.
El sub-bloque ShiftRegister se encarga de desplazar 1 posición los bits recibidos a través de la entrada *D,* comprobando el valor de la entrada *Enable* para ver si son válidos. Saca como salida, cada ciclo de reloj, una palabra de longitud *WrdLngth* a través del puerto *Q,* que llega al sub-bloque FIFO.
En el sistema implementado la palabra es de 8 bits, pero se generaliza para cualquier caso mediante *WrdLenght*.
El sub-bloque FIFO recibe de ShifRegister la palabra a través de la entrada *Din* y la orden de escritura en memoria a través de la entrada *Wr_en*, que viene del sub-bloque RS232_RX cuando ha validado tantos bits como grande sea el tamaño de palabra, entonces, se guarda la palabra en la FIFO. Además, recibe una orden de lectura de palabra a través de la entrada *Rd_en*, que permite obtener como salida del bloque RS232_RX la última palabra guardada a través de *Data_out*. A su vez, contiene líneas para avisar de su estado, llena (salida *Full*) o vacía (salida *Empty*).
Como se observa según la máquina de estados, el bloque se encuentra en su estado *Idle* hasta que recibe el bit de inicio activo a nivel bajo, entonces pasa al estado *StartBit*. En este estado, se adecúa el tamaño de bit según la frecuencia de reloj y la velocidad de transmisión escogida, contando ciclos de reloj y almacenándolos en la variable *BitCounter*. Cuando el contador *BitCounter* llega al final, pasa a siguiente estado *RcvData*, en él se muestrea la señal a mitad de cuenta según el contador *HalfBitCounter*, si este bit es válido, se pone a ‘1’ la salida *Valid_out* y se aumenta en 1 el valor del contador *DataCount*. El proceso se repite hasta llegar al tamaño de palabra determinado, momento en el que pasa al estado *StopBit.* Cuando *BitCounter* completa su cuenta, vuelve al estado inicial *Idle*, el que se pone la salida *Store_out* a ‘1’ para guardar esa palabra en la memoria FIFO.

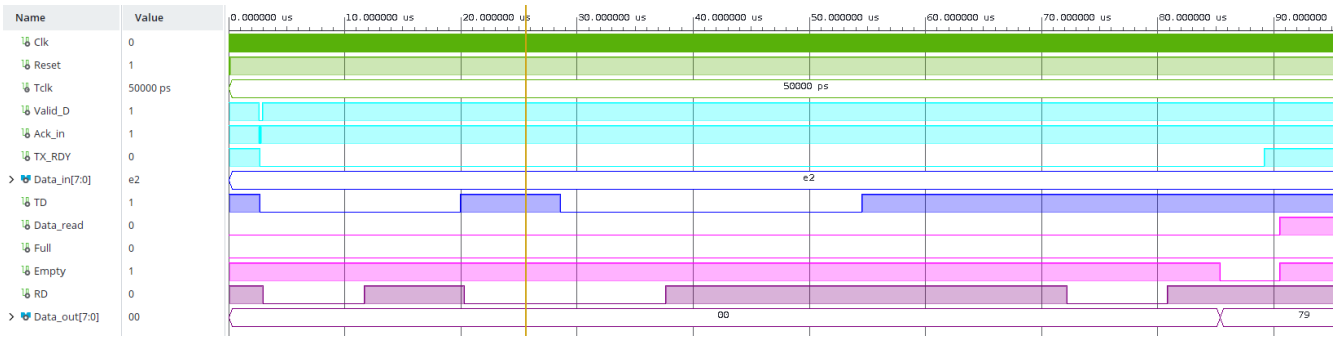




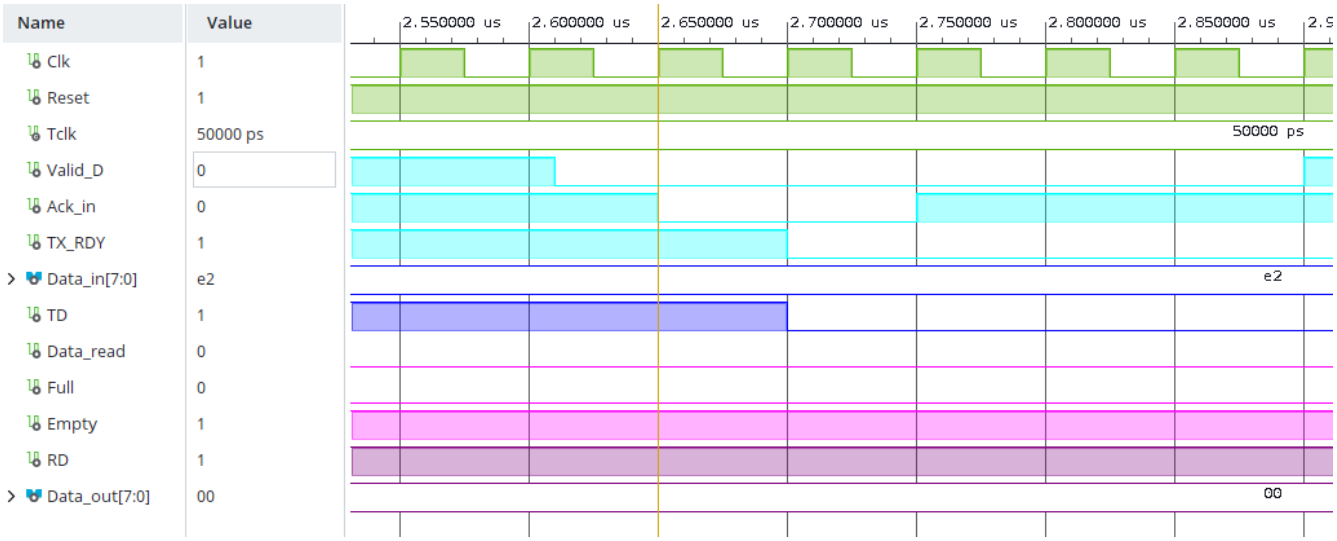
Por su parte, el bloque RS232_TX gestiona la transmisión de datos desde el cliente hacia el exterior. Consta de reloj *Clk* y entrada *Reset* asíncrona.

Recibe la palabra a transmitir y la señal *Start* de inicio de transmisión, momento en el que pasa del estado *Idle* al estado *StartBit.* En él cuenta ciclos de reloj hasta llegar a la longitud adecuada de bit, según la frecuencia de reloj y la velocidad de transmisión elegida. A continuación, pasa al estado *SendData*, donde envía uno a uno todos bits de la palabra a través de la salida *TX.* Cuando ha terminado, pasa al estado *StopBit*, dónde envía el bit de parada con el tamaño adecuado, para finalmente volver al estado *Idle*, dónde pone la salida *EOT* a '1' para indicar que la transmisión ha finalizado y se puede volver a usar esta línea, en cualquier otro estado se encuentra ocupada, indicado mediante *EOT* puesta a '0'.
En la implementación en placa, la señal *Valid_D* proviene del botón BTNC de la placa, que activa la señal *Start* que inicia la transmisión, pero en otro caso, esta señal debería provenir desde un sistema cliente externo. La señal *ACK_in* indica cuándo un bit se está empleando para su transmisión.

2.2) Simulaciones y prueba en placa del RS-232



Se ha comprobado su correcto funcionamiento mediante simulación haciendo uso del testbench *tb_RS232top* facilitado en Moodle según se observa las gráficas de salida de la primera Figura, sin embargo, no se ha podido confirmar su correcto funcionamiento en placa. Tras haberlo intentado y comprobado el código, se ha llegado a la siguiente conclusión: en el proceso *Clocking* de la entidad RS232top ocurre un hecho extraño en la temporización que involucra las señales *Valid_D* y *TX_RDY_i.*
La señal *Valid_D* se corresponde con el botón BTNC, de tal manera que si esté se pulsa la señal vale '0', por otra parte, la señal *TX_RDY_i* se corresponde con la señal *EOT* del módulo RS232_TX, siendo su valor '1' sólo para el estado *Idle.* De manera predeterminada, el módulo RS232_TX se encuentra en este estado, por tanto, al desactivar el *Reset* asociado a SW (15) = '1', al configurar un dato mediante los switches SW (7…0) y pulsar el botón BTNC, debería comenzar la transmisión, sin embargo, aunque el LED (11) = *TX_RDY* se encuentre encendido, esto no sucede y el LED (10) = *Ack_in* no se apaga, indicando que no se cumple la condición *if Valid_D = '0' and TX_RDY_i = '1'*, como sí sucede en la simulación durante al menos un ciclo de reloj, como se observa la siguientes Figura. Se desconoce el motivo de este comportamiento.



```
-- purpose: Clocking process for input protocol
Clocking : process (Clk, Reset)
begin
  if Reset = '0' then  -- asynchronous reset (active low)
    Data_FF   <= (others => '0');
    LineRD_in <= '1';
    Ack_in    <= '1';
  elsif rising_edge(Clk) then  -- rising edge of the clock
    LineRD_in <= RD;
    if Valid_D = '0' and TX_RDY_i = '1' then
      Data_FF <= Data_in;
      Ack_in  <= '0';
      StartTX <= '1';
    else
      Ack_in  <= '1';
      StartTX <= '0';
    end if;
  end if;
end process Clocking;

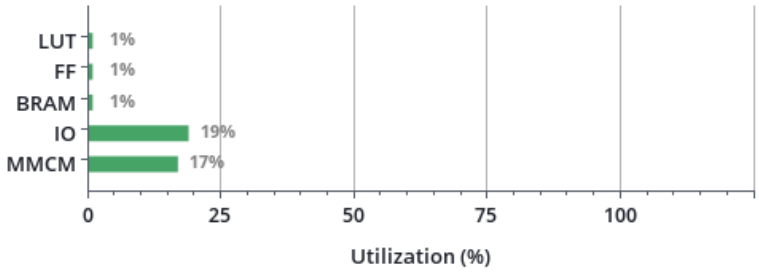
TX_RDY <= TX_RDY_i;
```

2.3) Resultados de la síntesis

Los resultados obtenidos son los expuesto en la figura. Parecen razonables porque los porcentajes de utilización de recursos son bajos, acordes a la solución implementada.

Summary

Resource	Utilization	Available	Utilization %
LUT	72	63400	0.11
FF	92	126800	0.07
BRAM	0.50	135	0.37
IO	39	210	18.57
MMCM	1	6	16.67



2.4) Mejoras del RS-232

No se han realizado mejoras de momento, primero se pretende hacer funcionar el sistema en placa, para después incluir estar mejoras. Entre ellas se pretende implementar la configuración externa del tamaño de palabra y la velocidad de transmisión.